

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 2 – REVIEW PENGENALAN PEMROGRAMAN



NAMA

NIM

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Rekursif adalah teknik dalam pemrograman di mana sebuah fungsi memanggil dirinya sendiri untuk menyelesaikan suatu masalah secara bertahap. Proses ini dilakukan dengan memecah masalah menjadi bagian yang lebih kecil hingga mencapai kondisi berhenti (base case). Jika tidak ada base case, maka program akan terus berjalan tanpa henti.

Rekursif terdiri dari dua bagian utama, yaitu base case dan recursive case. Base case berfungsi untuk menghentikan proses rekursif, sedangkan recursive case adalah bagian ketika fungsi memanggil dirinya sendiri dengan nilai yang semakin mendekati kondisi berhenti. Dengan cara ini, masalah yang kompleks dapat diselesaikan secara bertahap.

Penggunaan rekursif memiliki kelebihan, yaitu membuat program lebih ringkas dan mudah dipahami, terutama untuk masalah yang memiliki pola berulang seperti faktorial dan deret Fibonacci. Namun, rekursif juga memiliki kekurangan karena membutuhkan lebih banyak memori akibat penggunaan call stack. Jika tidak dirancang dengan baik, rekursif dapat menyebabkan stack overflow. Contoh penggunaan rekursif antara lain pada faktorial, Fibonacci, pencarian biner, dan traversal tree.

B. Guided

1. Guided 1

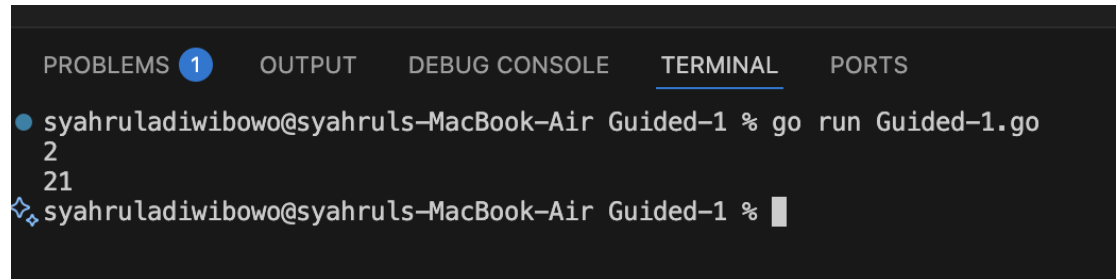
```
package main
import "fmt"

func tampilFaktor(n, i int) {
    if i > n {
        return
    }
    if n%i == 0 {
        fmt.Print(i, " ")
    }
    tampilFaktor(n, i+1)
}

func main() {
    var n int
    fmt.Print("Masukkan angka: ")
    fmt.Scan(&n)
```

```
fmt.Print("Faktor: ")
tampilFaktor(n, 1)
}
```

Output :

A screenshot of a Go IDE terminal window. The terminal has tabs for PROBLEMS (1), OUTPUT, DEBUG CONSOLE, TERMINAL (selected), and PORTS. The command prompt shows the user 'syahruladiwibowo@syahruls-MacBook-Air' in the 'Guided-1' directory, running 'go run Guided-1.go'. The output of the program is '2' followed by '21' on the next line. The terminal cursor is at the end of the command line.

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
● syahruladiwibowo@syahruls-MacBook-Air Guided-1 % go run Guided-1.go
2
21
❖ syahruladiwibowo@syahruls-MacBook-Air Guided-1 %
```

2. Guided 2

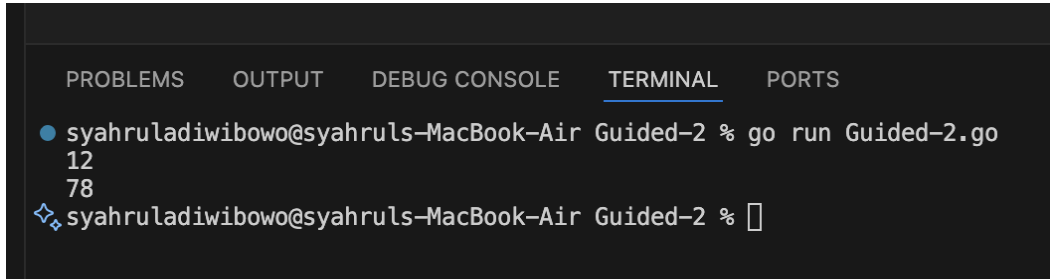
```
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)
    fmt.Println(penjumlahan(n))
}

func penjumlahan(n int) int {
    if n == 1 {
        return 1
    } else {
        return n + penjumlahan(n-1)
    }
}
```

Output :



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● syahruladiwibowo@syahruls-MacBook-Air Guided-2 % go run Guided-2.go
12
78
❖ syahruladiwibowo@syahruls-MacBook-Air Guided-2 %
```

3. [judul guided 3]

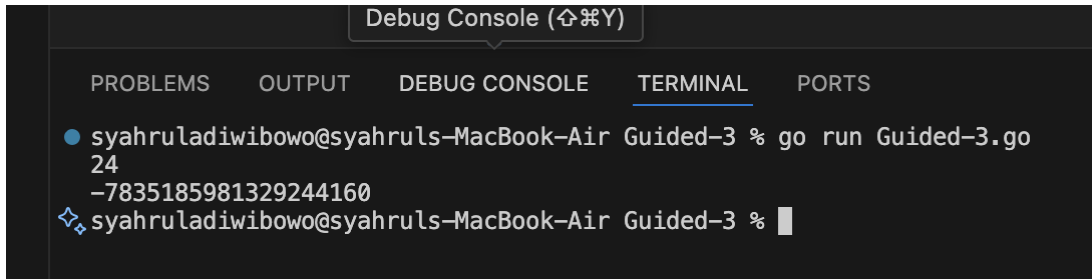
```
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)
    fmt.Println(pangkat(n))
}

func pangkat(n int) int {
    if n == 0 {
        return 1
    } else {
        return n * pangkat(n-1)
    }
}
```

Output :



```
Debug Console (⇧⌘Y)
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● syahruladiwibowo@syahruls-MacBook-Air Guided-3 % go run Guided-3.go
24
-7835185981329244160
❖ syahruladiwibowo@syahruls-MacBook-Air Guided-3 %
```

C. Unguided

1. Soal Unguided 1

- 1) Deret fibonacci adalah sebuah deret dengan nilai suku ke-0 dan ke-1 adalah 0 dan 1, dan nilai suku ke-n selanjutnya adalah hasil penjumlahan dua suku sebelumnya. Secara umum dapat diformulasikan $S_n = S_{n-1} + S_{n-2}$. Berikut ini adalah contoh nilai deret fibonacci hingga suku ke-10. Buatlah program yang mengimplementasikan fungsi rekursif pada deret fibonacci tersebut.

n	0	1	2	3	4	5	6	7	8	9	10
S_n	0	1	1	2	3	5	8	13	21	34	55

Jawaban :

```
package main

import "fmt"

func fibo(n int) int {
    if n == 0 {
        return 0
    }
    if n == 1 {
        return 1
    }
    return fibo(n-1) + fibo(n-2)
}

func main() {
    var n int
    fmt.Print("Input n: ")
    fmt.Scan(&n)

    fmt.Print("Hasil: ")
    for i := 0; i <= n; i++ {
        fmt.Print(fibo(i), " ")
    }
}
```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● syahruladiwibowo@syahruls-MacBook-Air Unguided-1 % go run Unguided-1.go
Input n: 12
Hasil: 0 1 1 2 3 5 8 13 21 34 55 89 144 %
✧ syahruladiwibowo@syahruls-MacBook-Air Unguided-1 %
```

Penjelasan :

Program ini menghasilkan deret Fibonacci menggunakan metode rekursif. Setiap nilai diperoleh dari penjumlahan dua nilai sebelumnya. Proses ini akan terus berjalan hingga mencapai kondisi dasar, yaitu saat n bernilai 0 atau 1. Program kemudian menampilkan seluruh deret dari 0 sampai n.

2. Soal Unguided 2

3) Buatlah program yang mengimplementasikan rekursif untuk menampilkan faktor bilangan dari suatu N, atau bilangan yang apa saja yang habis membagi N.

Masukan terdiri dari sebuah bilangan bulat positif N.

Keluaran terdiri dari barisan bilangan yang menjadi faktor dari N (terurut dari 1 hingga N ya).

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	5	1 5

Jawaban :

```
package main
import "fmt"

func tampilFaktor(n, i int) {
    if i > n {
        return
    }
    if n%i == 0 {
        fmt.Print(i, " ")
    }
    tampilFaktor(n, i+1)
}

func main() {
    var n int
    fmt.Print("Masukkan angka: ")
}
```

```

fmt.Scan(&n)

fmt.Print("Faktor: ")
tampilFaktor(n, 1)
}

```

Output :

```

syahruladiwibowo@syahruls-MacBook-Air Unguided-2 % go run Unguided-1.go
Masukkan angka: 12
Faktor: 1 2 3 4 6 12
syahruladiwibowo@syahruls-MacBook-Air Unguided-2 %

```

Penjelasan :

Program ini digunakan untuk menentukan faktor dari suatu bilangan dengan pendekatan rekursif. Fungsi akan memeriksa setiap bilangan mulai dari 1 hingga n. Jika suatu angka dapat membagi n tanpa sisa, maka angka tersebut akan ditampilkan sebagai faktor.

3. Soal Unguided 3

5) Buatlah program yang mengimplementasikan rekursif untuk menampilkan barisan bilangan ganjil.

Masukan terdiri dari sebuah bilangan bulat positif N.

Keluaran terdiri dari barisan bilangan ganjil dari 1 hingga N.

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	5	1 3 5
2	20	1 3 5 7 9 11 13 15 17 19

Jawaban :

```

package main

import "fmt"

func tampilGanjil(n int) {
    if n < 1 {
        return
    }
}

```

```

tampilGanjil(n - 1)

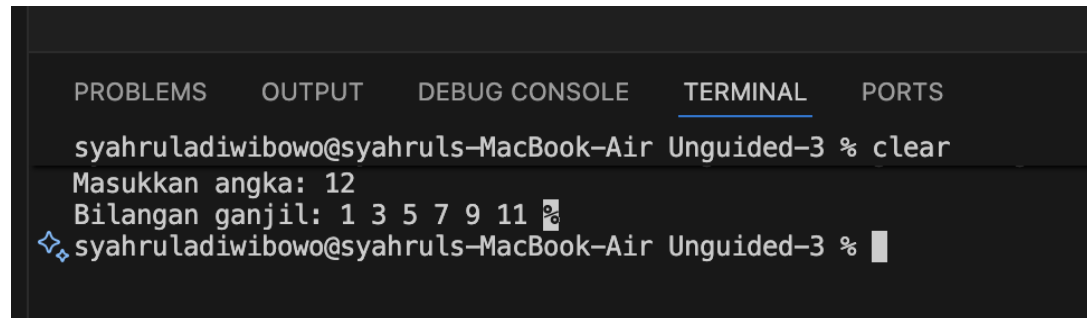
if n%2 != 0 {
    fmt.Print(n, " ")
}

func main() {
    var n int
    fmt.Print("Masukkan angka: ")
    fmt.Scan(&n)

    fmt.Print("Bilangan ganjil: ")
    tampilGanjil(n)
}

```

Output :



```

syahruladiwibowo@syahruls-MacBook-Air Unguided-3 % clear
Masukkan angka: 12
Bilangan ganjil: 1 3 5 7 9 11 %
syahruladiwibowo@syahruls-MacBook-Air Unguided-3 %

```

Penjelasan :

Program ini menampilkan bilangan ganjil dari 1 hingga n menggunakan rekursif. Fungsi akan dipanggil berulang dengan nilai yang semakin kecil, kemudian saat proses kembali, program mengecek apakah bilangan tersebut ganjil. Jika iya, maka akan ditampilkan.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa rekursif merupakan metode pemrograman yang digunakan untuk menyelesaikan masalah dengan cara memanggil fungsi secara berulang hingga mencapai kondisi berhenti.

Rekursif sangat membantu dalam menyelesaikan permasalahan yang memiliki pola berulang, seperti faktorial dan deret Fibonacci.

Namun, penggunaan rekursif harus dirancang dengan baik, terutama dalam menentukan base case, agar tidak terjadi perulangan tanpa akhir. Selain itu, rekursif juga membutuhkan penggunaan memori yang lebih besar karena adanya call stack. Oleh karena itu, rekursif sebaiknya digunakan pada kasus yang sesuai agar program tetap efisien dan mudah dipahami.